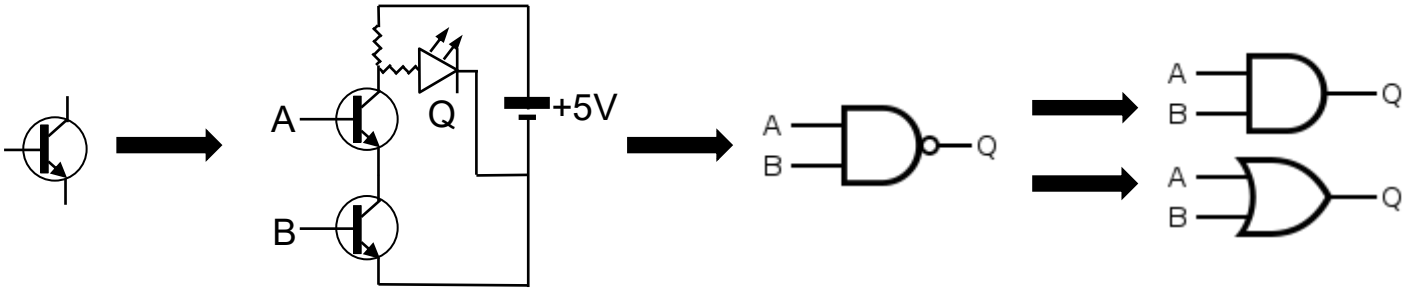
COSC204
Computer Systems

Sequential Circuits

Overview

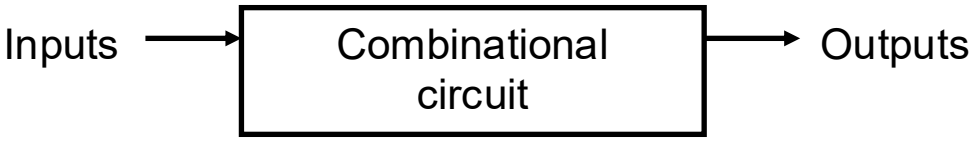
- Last Lecture



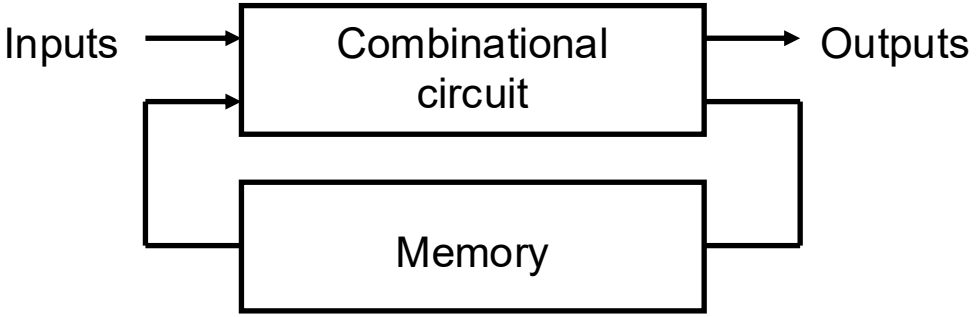
- This Lecture
  - Sequential circuits
  - Source: Chapter 12 (11th edition)
- Next Lecture
  - Computer architecture

Introduction

- ***Combinatorial logic circuit*** - the same inputs always produces the same outputs (last lecture)



- ***Sequential logic circuit*** - outputs depend not only on the present inputs, but also on the past history (this lecture)



Introduction

- Types of Sequential Circuits
  - Synchronous
    - State changes occur in synchronization to a clock signal
    - Input changes occur between clock pulses
    - State changes occur at the clock pulses
  - Asynchronous
    - ***We will not be studying these***
    - State changes occur as changes in inputs occur
    - Event driven

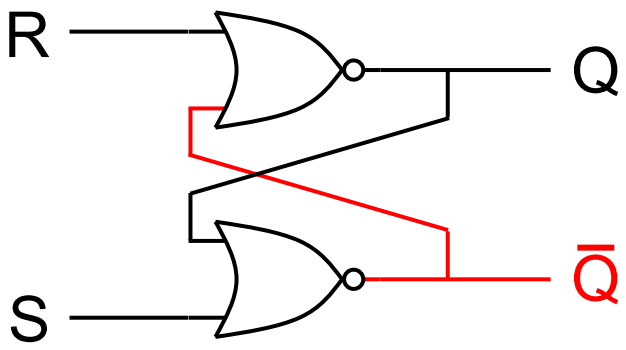
Memory Circuits

- Set-reset latch
- D-type flip-flop
- T-type flip-flop
- JK-type flip-flop

Set-Reset Latch

- If you tie the output of the circuit to one of its inputs then the circuit saves state. In this case ***output is on Q***

- Start with R = S = Q = 0 ($\overline{Q}$= 1), the latch stores a 0



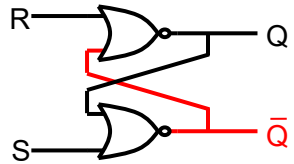
NOR

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

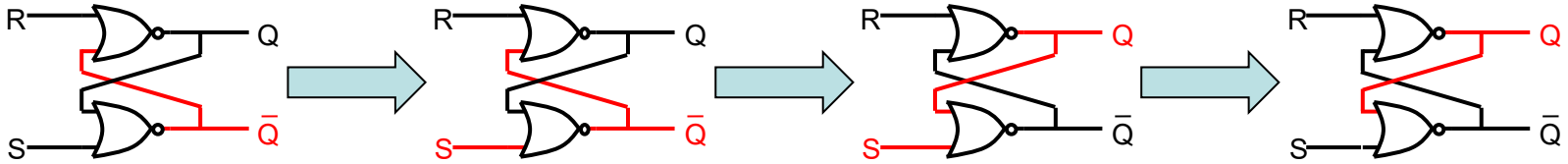
Set (Q) the SR Latch

NOR		
A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

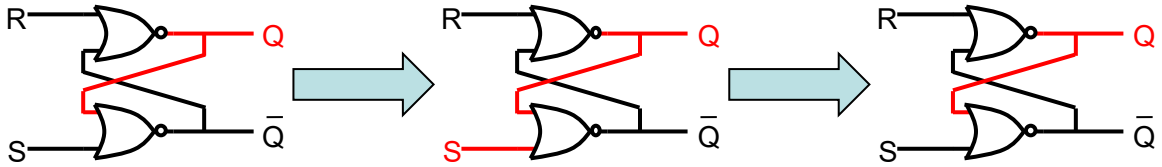
- Q = 0



- Now toggle S = 1 (set the latch)



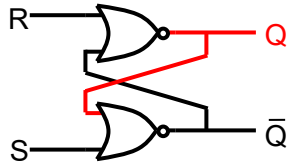
- Q = 1, Now toggle S = 1 (set the latch)



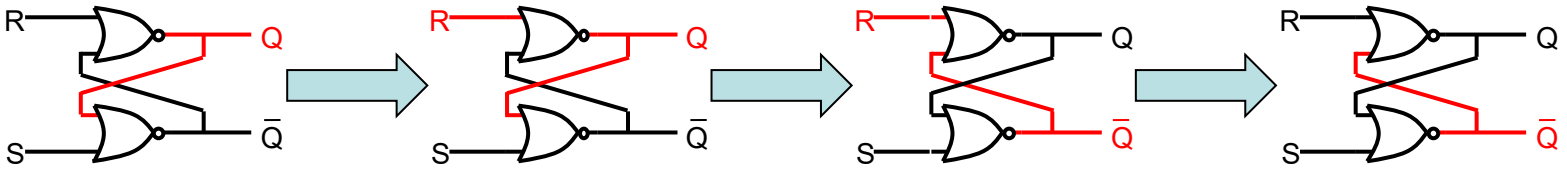
Reset (Q) SR Latch

NOR		
A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

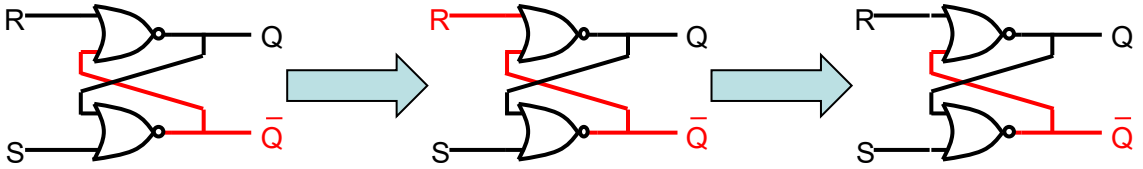
- Q = 1



- Now toggle R = 1 (reset the latch)



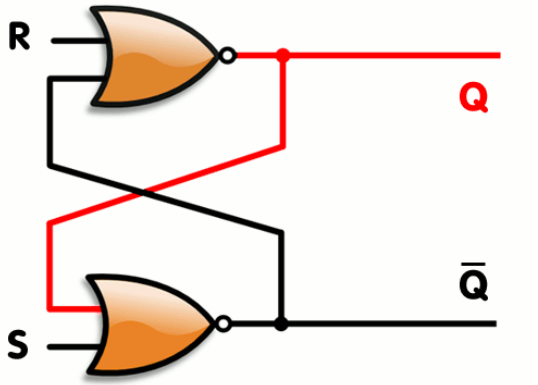
- Q = 0, Now toggle R = 1 (reset the latch)



Set-Reset Latch

- Regardless of Q,
  - By setting S = 1 the latch is set (Q = 1)
  - By setting R = 1 the latch is reset (Q = 0)

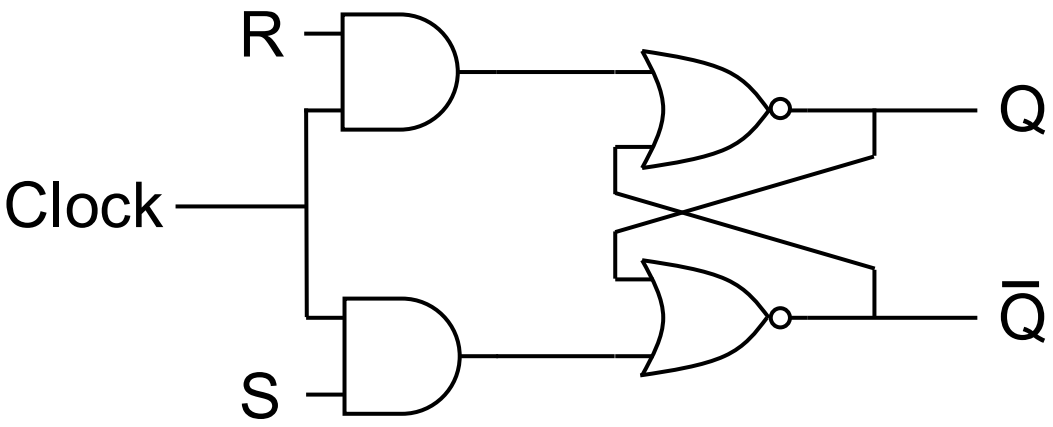
S	R	Action
0	0	None
0	1	Q=0
1	0	Q=1
1	1	Invalid (why?)



From: http://en.wikipedia.org/wiki/File:R-S_mk2.gif

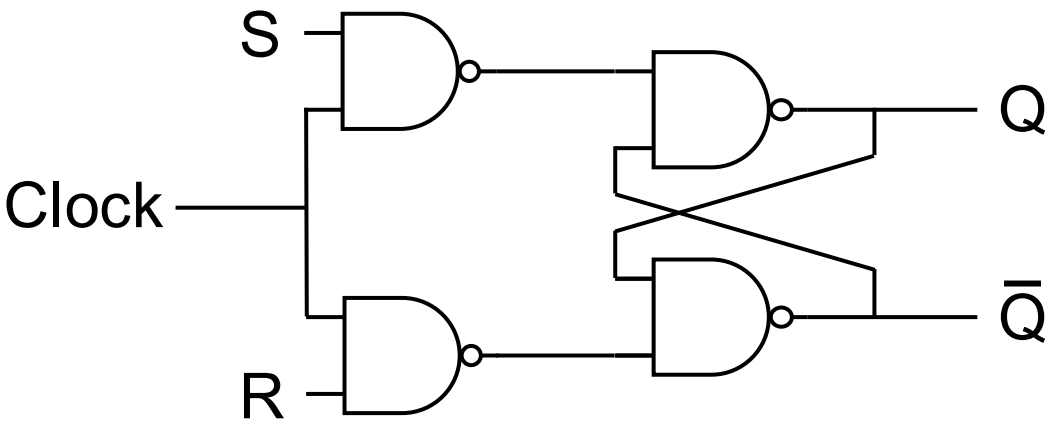
Clocked (*Enabled*) SR Flip-Flop

- The SR latch is a nice trick
- If we add a control line to the circuit then we can turn the latch on and off when we need it
  - The control line is normally referred to as a “clock” line



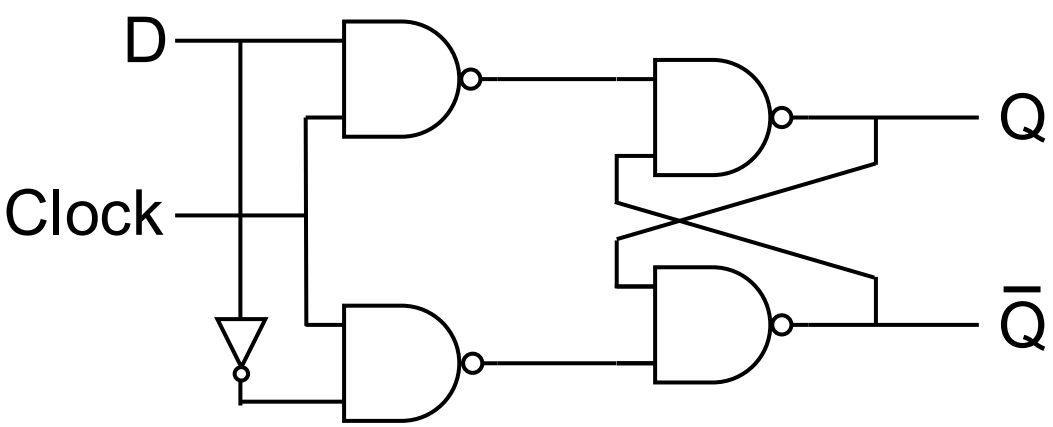
Clocked SR NAND flip-flop

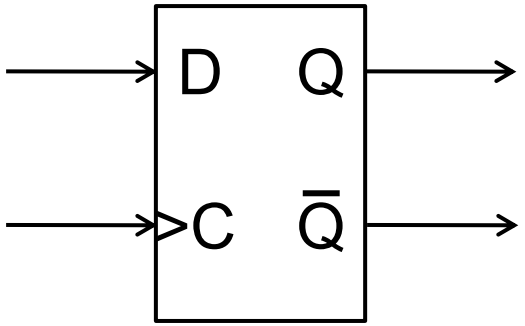
- By inverting the signals into the SR latch, a flip-flop can be built using only NAND gates (De Morgan)
  - And recall that a NAND gate can be built with 2 transistors



D-Type Flip-Flop

- Since S = $\bar{R}$, we can combine them into one line: D



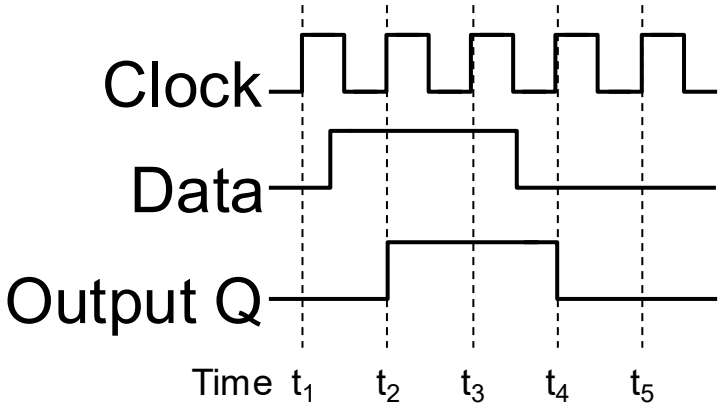


D-Type flip-flop

Triangle indicates ‘edge-triggered’ clock activation, but the figure on the left uses ‘level-triggered’ clock activation.

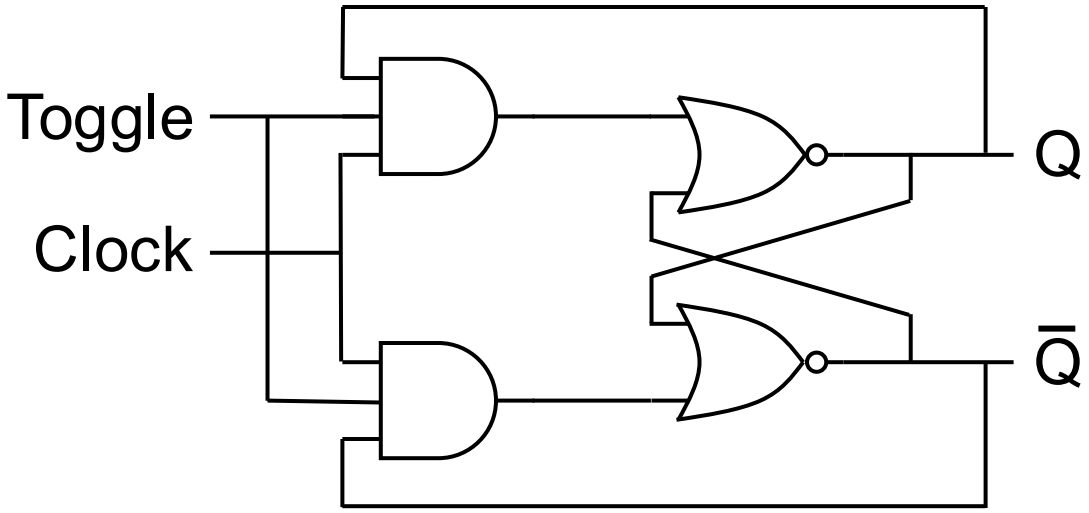
D-Type Flip-Flop

- Data flip-flop (or Delay flip-flop)
  - Set or reset D
  - Toggle the clock line (from 0 to 1)
    - D is stored (at that clock rising edge-triggered moment)
  - Toggle the clock line (from 1 to 0)
  - D can vary (wobble) freely
- There is up to a 1 clock-cycle delay between D and Q



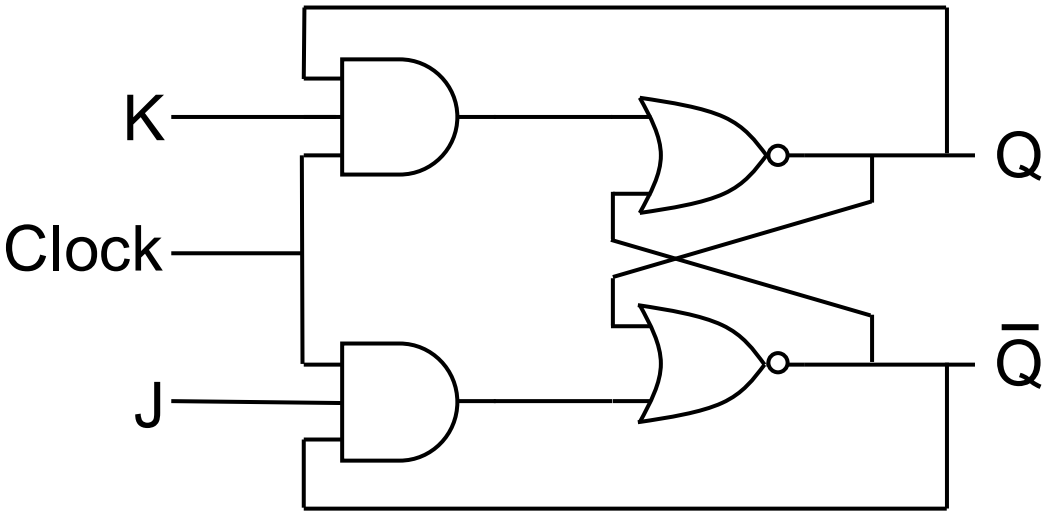
T Flip-Flop

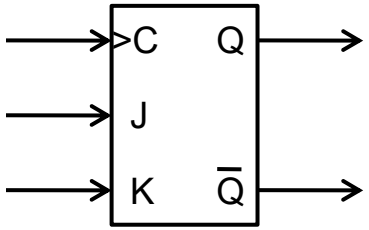
- Toggle Flip-Flop
  - The output is inverted when:
    - Toggle is held high
    - Clock line is cycled (from off to on to off)



JK Flip-Flop

- The one flip-flop to rule them all (the universal flip-flop)



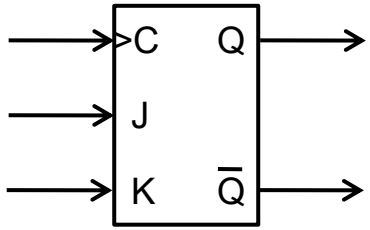


JK flip-flop

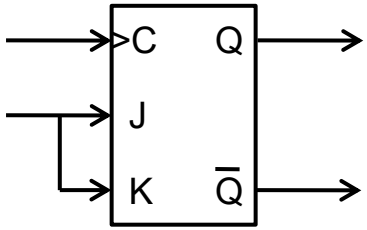
J	K	Q_{t+1}	Meaning
0	0	Q_t	No Change
0	1	0	Reset
1	0	1	Set
1	1	$\overline{Q}_t$	Toggle

JK Flip-Flop

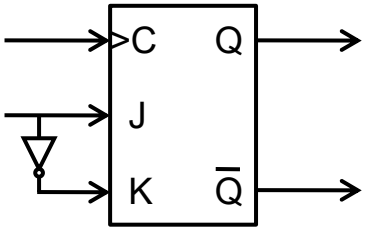
- One flip-flop to rule them all (the universal flip-flop)
  - Set K = J to get T flip-flop
  - Set K = $\overline{J}$ to get a D flip-flop



JK flip-flop



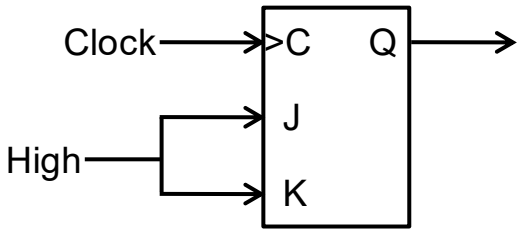
T flip-flop

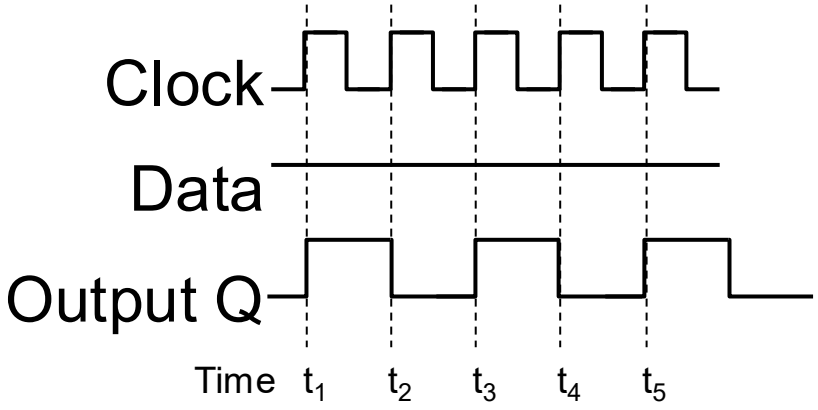


D flip-flop

Asynchronous Ripple Counter

- A 1-bit counter switches between 0 and 1 each cycle
  - It's a toggle flip-flop
  - It's also a divide by 2

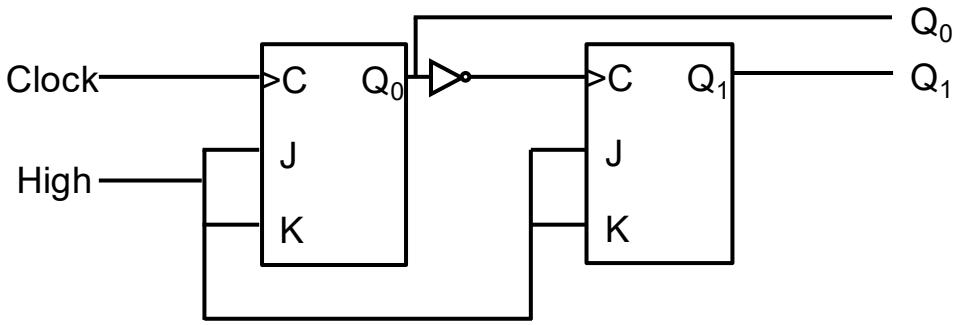




The output square wave is half the frequency of the input square wave

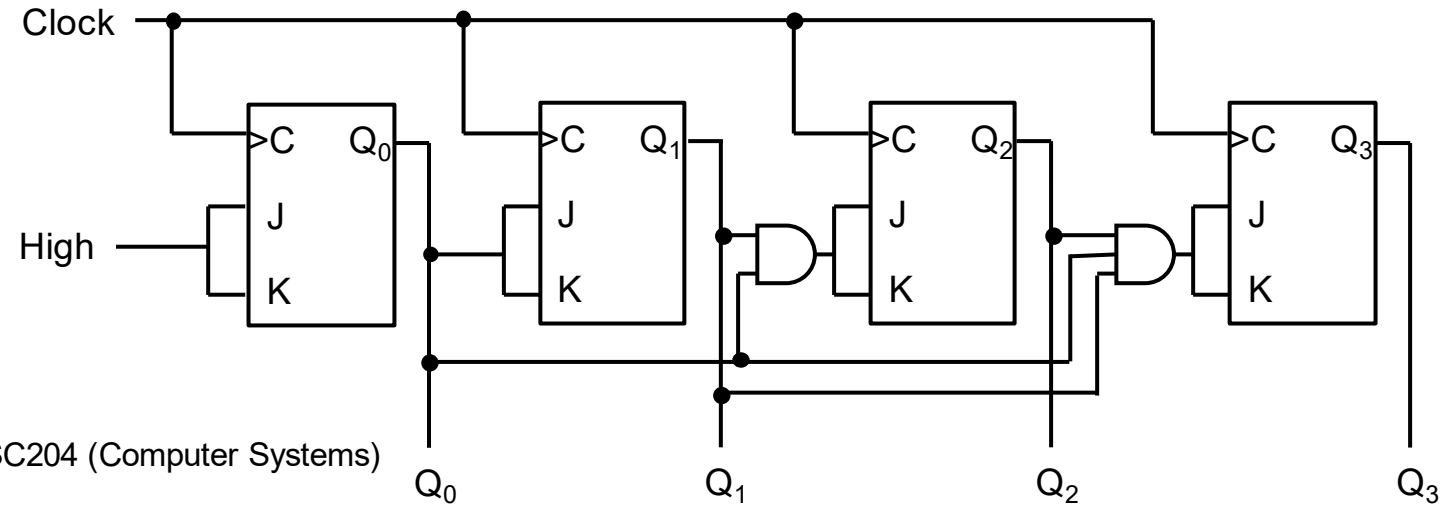
Asynchronous Ripple Counter

- A 2-bit counter
  - The low bit toggles every full clock cycle
  - The high bit toggles at half the clock cycle rate
    - Tie the output of the low bit to the input of the high bit



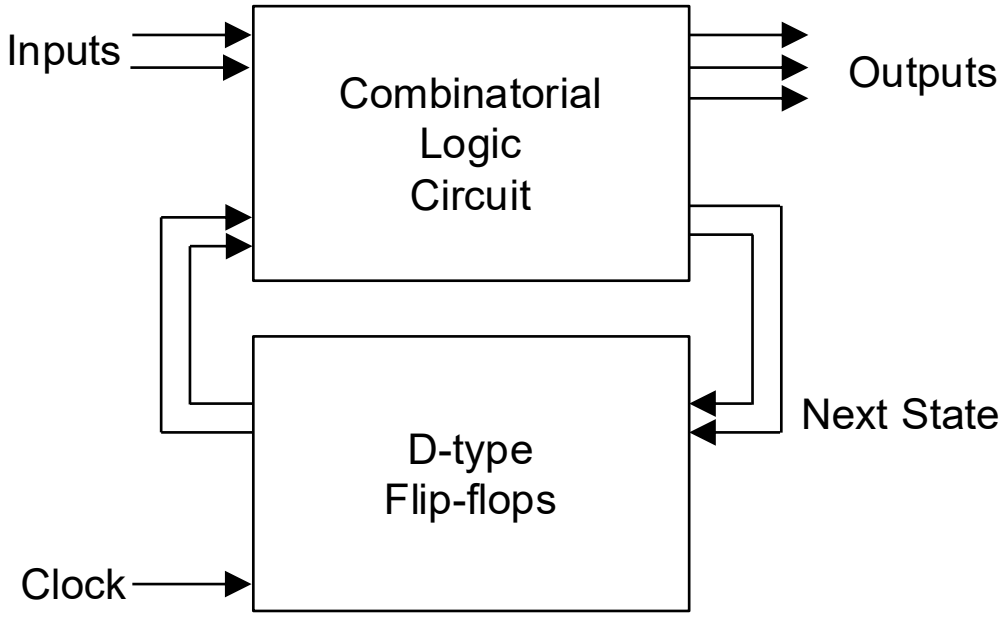
Synchronous Counter

- The asynchronous counter has the problem that the propagation delay is inconsistent; a function of the number of transistors the signal passes through
- Ideally the flip-flops will clock at the same time
  - This is achieved with a synchronous counter
  - Logic is added to set the next state based on the previous state

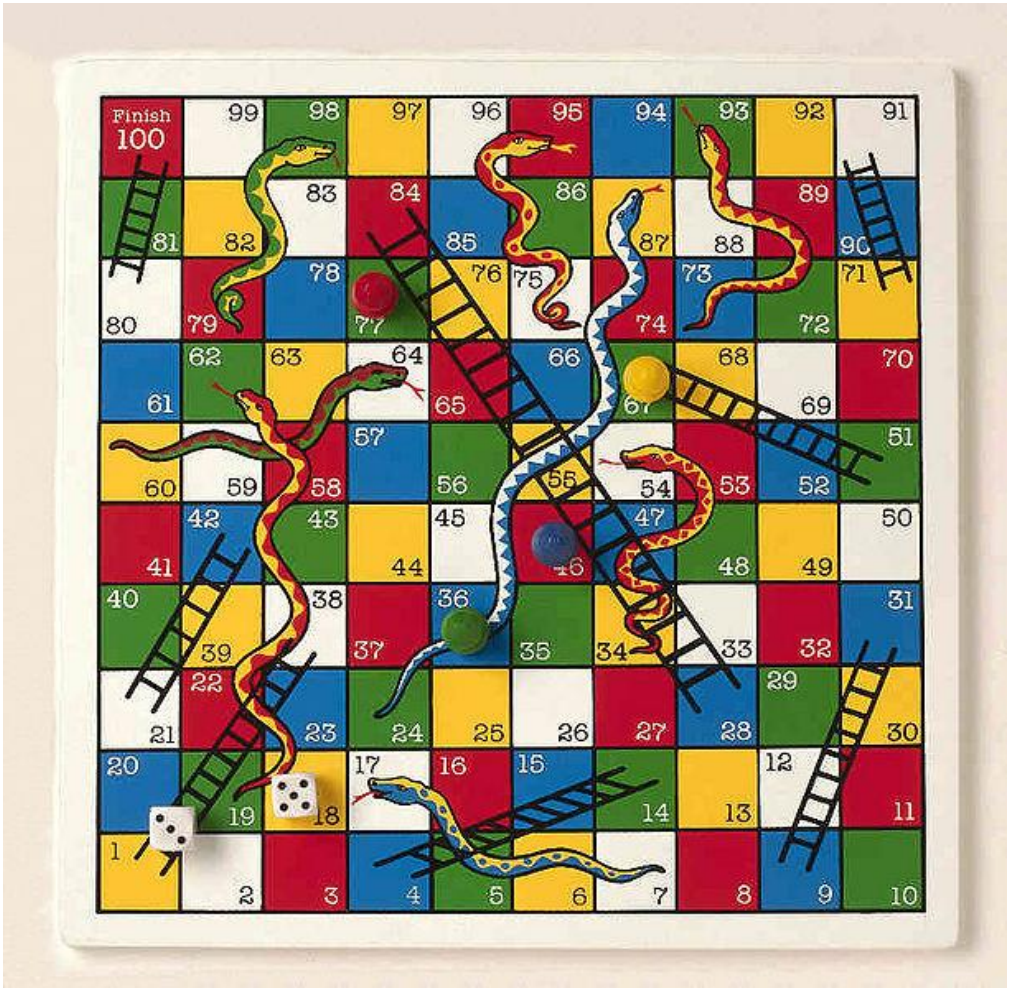


Sequential Circuits

- Sequential logic circuits store state between each use (i.e. they have memory)

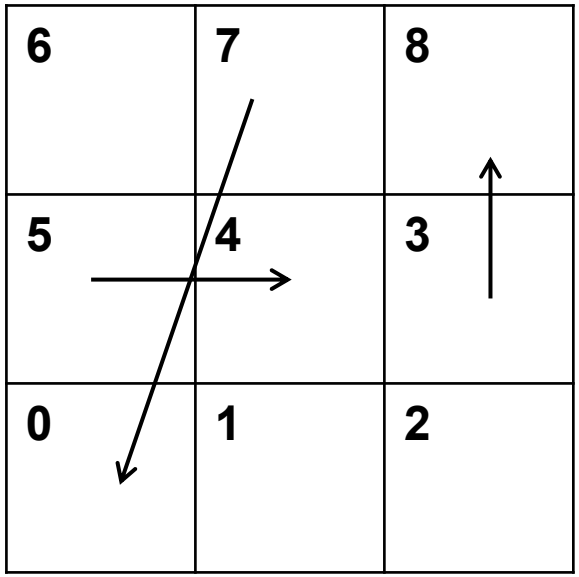


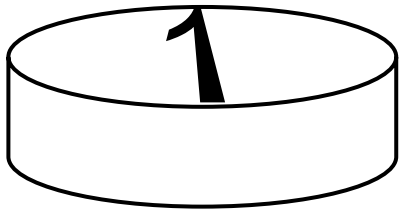
Snakes and Ladders



From: https://wiki.sfu.ca/personal/aoberman/images/thumb/b/b5/SnakesAndLadders.jpg/609px-SnakesAndLadders.jpg

Binary Snakes and Ladders

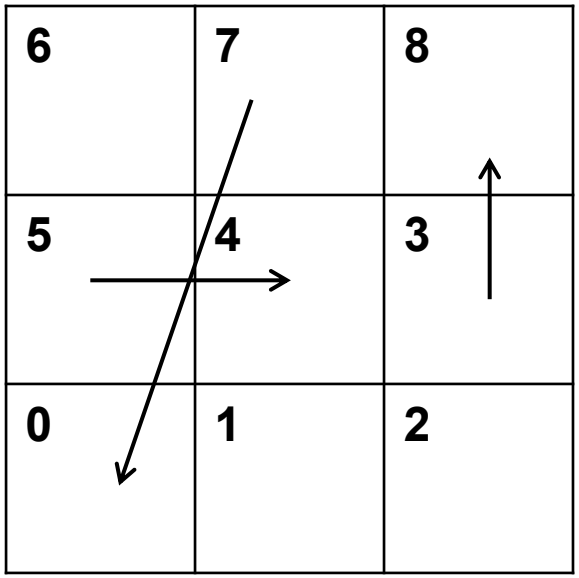




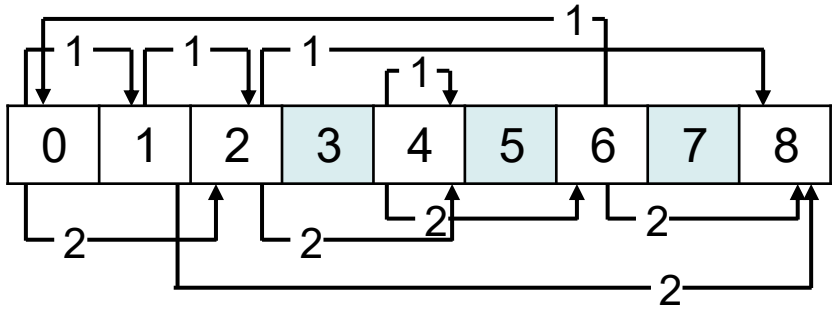
1 = move 1
2 = move 2

In this case a 3 × 3 board and one binary die (1 or 2)

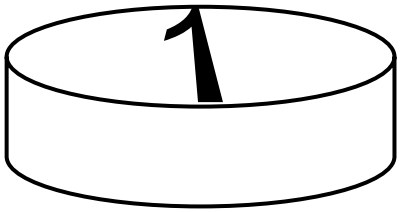
Binary Snakes and Ladders



Board



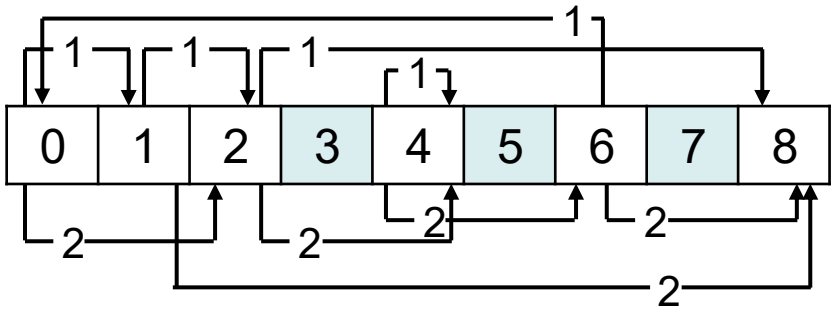
State Transition Diagram



1 = move 1
2 = move 2

State Transition Diagram

- The *State Transition Diagram* says "If you are here and you get that then go there"
- This is the basis of all computing (think: Turing Machine)
- The computer memory is the current state, the instruction is where to go next



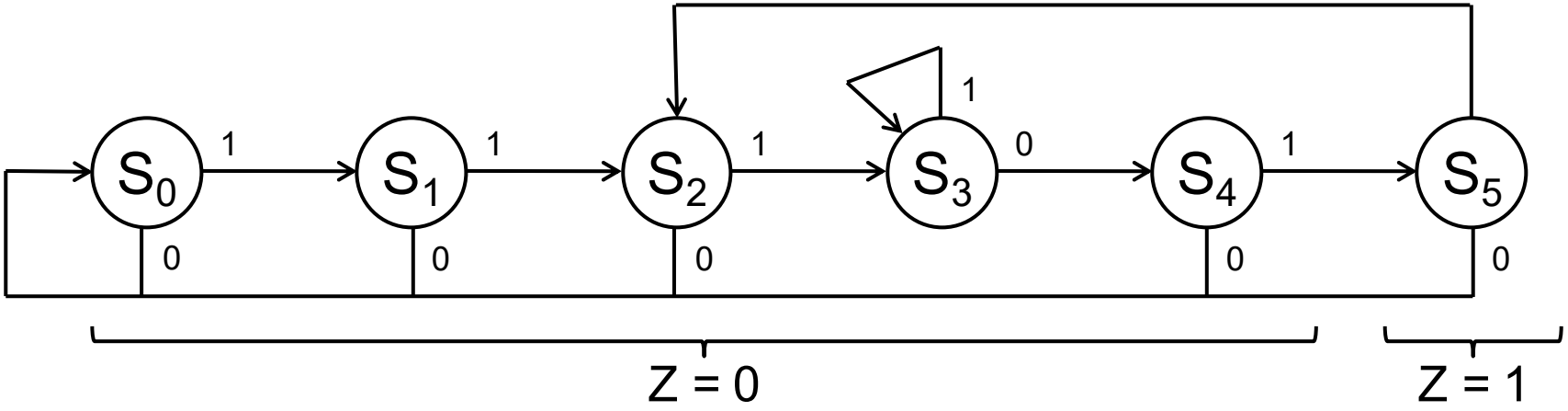
- Let's see another example…

Detect This Input Sequence

- Detect the input sequence 11101
- Single input X
- Single output Z
- Consider repeating sequences
  - E.g. 11111111101

- We're going to build the circuit!

State Transition Diagram



Detect This Input Sequence

- Draw the state table

Input	Present State	Next State	Output
0	S0	S0	0
1	S0	S1	0
0	S1	S0	0
1	S1	S2	0
0	S2	S0	0
1	S2	S3	0
0	S3	S4	0
1	S3	S3	0
0	S4	S0	0
1	S4	S5	0
0	S5	S0	1
1	S5	S2	1

Detect This Input Sequence

- 6 states → 3 state variables (to count to 6 in binary)
  - A, B, C
- Using A, B, C (in that order), let
  - S0 = 0 0 0
  - S1 = 0 0 1
  - S2 = 0 1 0
  - S3 = 0 1 1
  - S4 = 1 0 0
  - S5 = 1 0 1
- Store the current state in 3 flip-flops

Detect This Input Sequence

- Complete state table

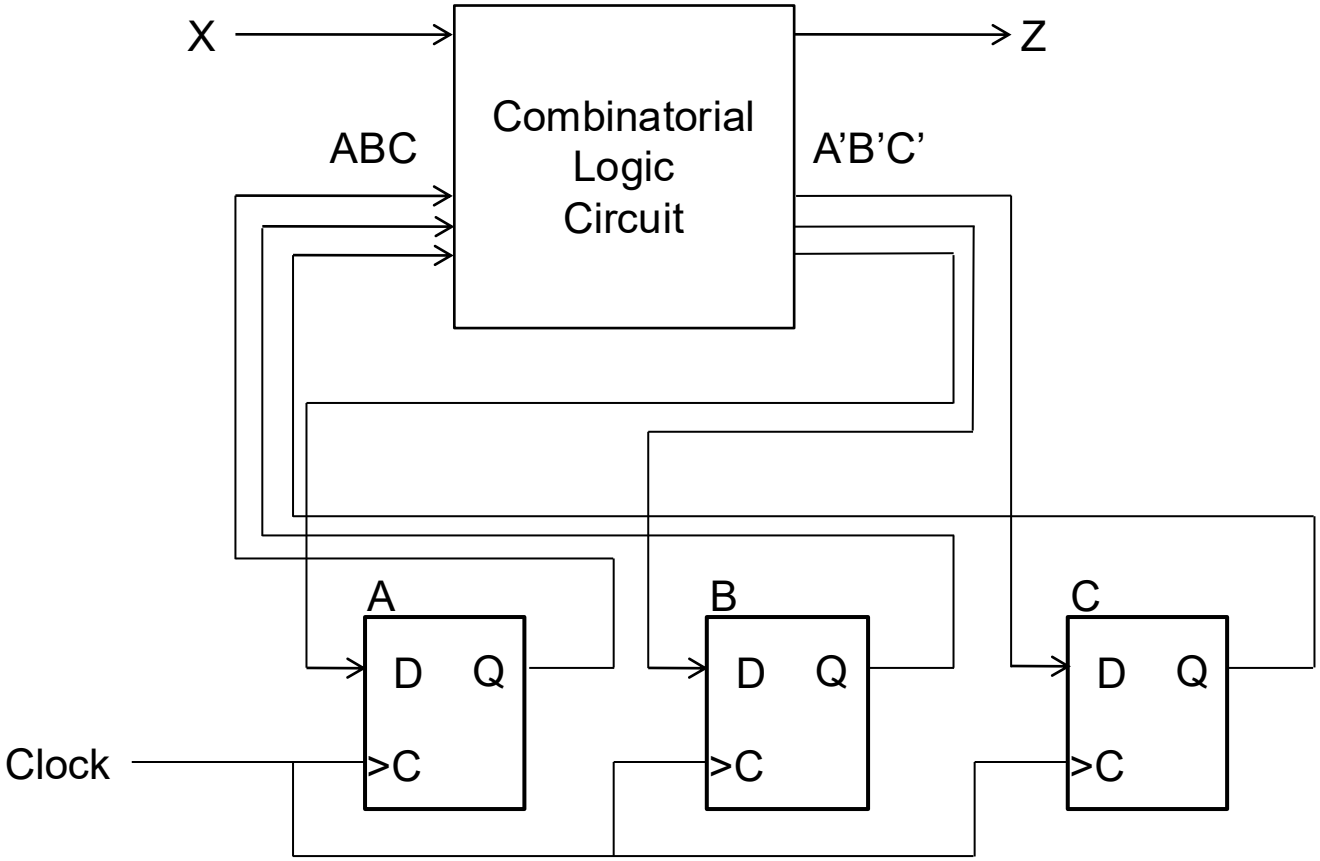
Present State			Input	Next State			Output
A	B	C	X	A'	B'	C'	Z
0	0	0	0	0	0	0	0
0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0
0	0	1	1	0	1	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	1	1	0
0	1	1	0	1	0	0	0
0	1	1	1	0	1	1	0
1	0	0	0	0	0	0	0
1	0	0	1	1	0	1	0
1	0	1	0	0	0	0	1
1	0	1	1	0	1	0	1

Detect This Input Sequence

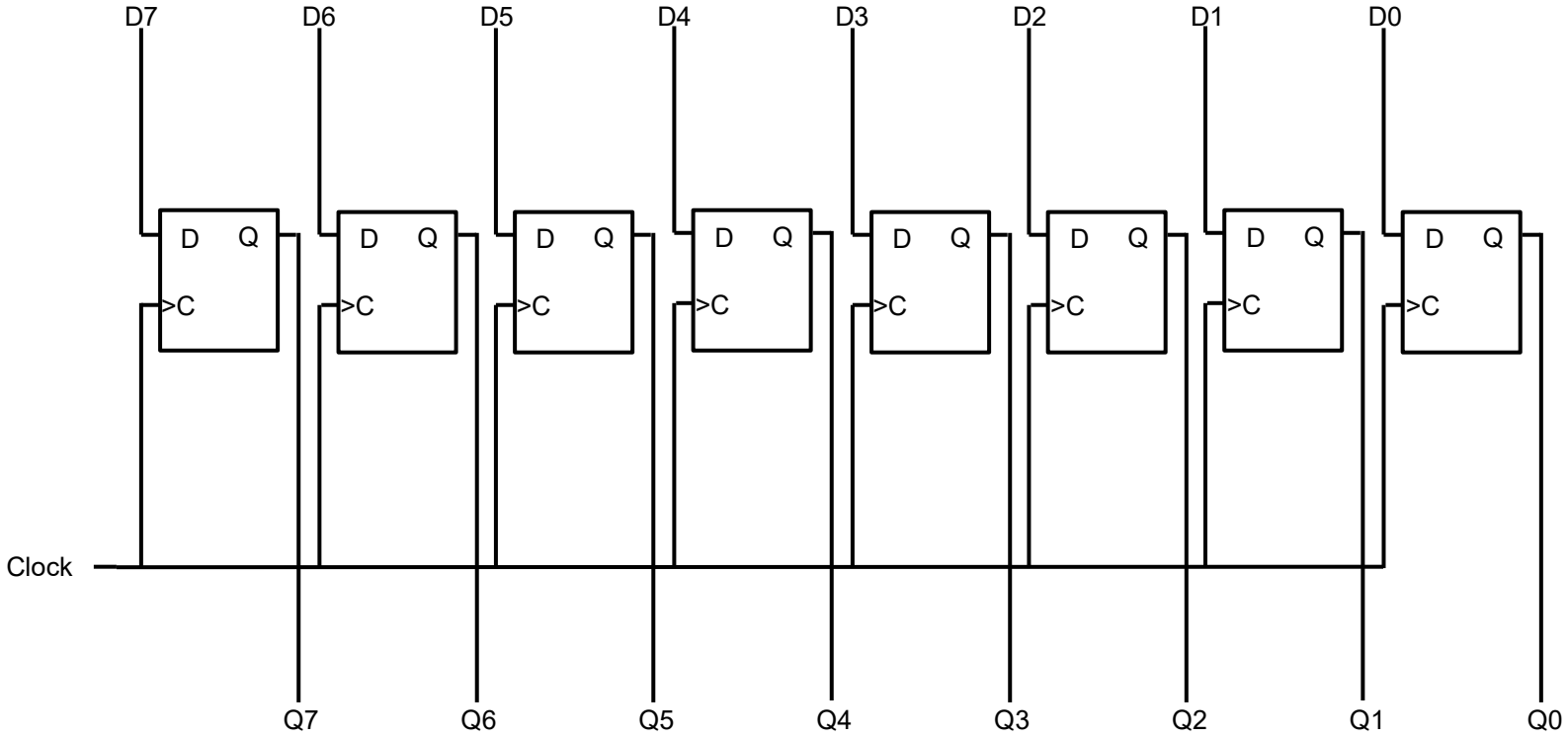
- Now we have a table just like we used in combinatorial logic circuits, so we can write down the logic for the stored state (A',B',C') and the output (Z)

- $A' = \overline{A}BC\overline{X} + A\overline{B}\overline{C}X$
- $B' = \overline{A}\overline{B}CX + \overline{A}B\overline{C}X + \overline{A}BCX + A\overline{B}CX$
- $C' = \overline{A}\overline{B}\overline{C}X + \overline{A}B\overline{C}X + \overline{A}BCX + A\overline{B}\overline{C}X$
- $Z = A\overline{B}C\overline{X} + A\overline{B}CX$

Detect This Input Sequence



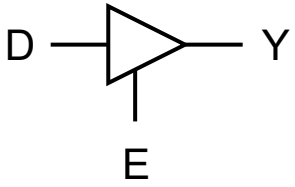
Registers

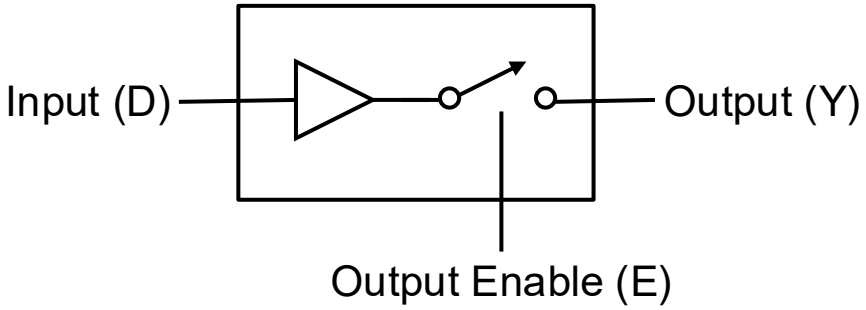


Except… we want to control when the register outputs a value

Tristate Buffer

- The clock is used to enable flip-flop load
- The output can be controlled with a 3-state buffer
  - Or a 3-state inverter
  - High, low, or “high impedance” (disconnected)

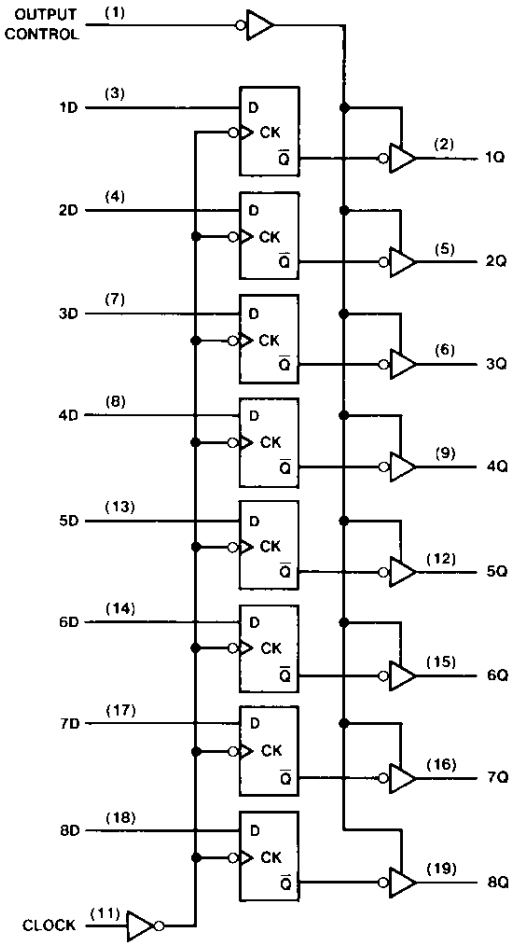




74HC374

- 3-STATE Octal D-Type Transparent Latches and Edge-Triggered Flip-Flops

- Output Control
  - "read line"
- Clock
  - "write line"



Homework

- Draw the circuit for the D-type flip-flop using only NOR gates
- How many gates are needed?
- Is this more or less than the NAND version?